

AWS S3 → Lambda → DynamoDB Metadata Pipeline

Objective

Build an automated pipeline where: 1. A file is uploaded to S3 2. S3 triggers a Lambda function 3. Lambda stores file metadata in DynamoDB

Architecture Overview

S3 (Object Upload) → Lambda (Event Trigger) → DynamoDB (Store Metadata)

Prerequisites

- AWS Account
 - IAM permissions to create S3, Lambda, DynamoDB, and IAM roles
 - Basic understanding of AWS Console
 - Python 3.x runtime knowledge
-

Step 1: Create DynamoDB Table

1. Go to AWS Console → DynamoDB
2. Click “Create Table”
3. Table Name: S3FileMetadata
4. Partition Key: FileName (String)
5. (Optional) Add Sort Key: EventTime (String)
6. Leave other settings as default
7. Click **Create**

Recommended Table Attributes

Attribute	Type
FileName	String
EventTime	String
BucketName	String
FileSize	Number

Attribute	Type
ETag	String

Step 2: Create IAM Role for Lambda

1. Go to IAM → Roles
2. Click **Create Role**
3. Select **AWS Service** → **Lambda**
4. Attach Policies:
 - AWSLambdaBasicExecutionRole
 - Custom policy (below)

Custom DynamoDB Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "dynamodb:PutItem"
      ],
      "Resource": "arn:aws:dynamodb:REGION:ACCOUNT_ID:table/S3FileMetadata"
    }
  ]
}
```

Replace: - REGION with your AWS region - ACCOUNT_ID with your AWS account ID

Name the role: LambdaS3DynamoDBRole

Step 3: Create Lambda Function

1. Go to AWS Console → Lambda
2. Click **Create Function**
3. Choose **Author from scratch**
4. Function Name: S3MetadataProcessor
5. Runtime: **Python 3.x**
6. Execution Role: Use existing role → LambdaS3DynamoDBRole

7. Click **Create**

Step 4: Configure Environment Variable (Recommended)

Go to: Configuration → Environment Variables → Add

Key	Value
TABLE_NAME	S3FileMetadata

Click **Save**.

Step 5: Add Lambda Code

Replace the default Lambda code with the following:

```
import json
import boto3
import urllib.parse
import os

# Initialize DynamoDB resource

dynamodb = boto3.resource('dynamodb')
TABLE_NAME = os.environ['TABLE_NAME']
table = dynamodb.Table(TABLE_NAME)

def lambda_handler(event, context):
    try:
        for record in event.get('Records', []):
            bucket = record['s3']['bucket']['name']

            key = urllib.parse.unquote_plus(
                record['s3']['object']['key'],
                encoding='utf-8'
            )

            item = {
                'FileName': key,
                'BucketName': bucket,
                'EventTime': record['eventTime'],
                'FileSize': record['s3']['object'].get('size', 0),
```

```
        'ETag': record['s3']['object'].get('eTag', '')
    }

    table.put_item(Item=item)
    print(f"Successfully stored metadata for {key}")

    return {
        'statusCode': 200,
        'body': json.dumps('Success')
    }

except Exception as e:
    print(f"Error: {str(e)}")
    raise e
```

Click **Deploy** after pasting the code.

Step 6: Create S3 Bucket

1. Go to AWS Console → S3
 2. Click **Create Bucket**
 3. Provide a globally unique bucket name
 4. Keep default settings (or adjust as needed)
 5. Click **Create Bucket**
-

Step 7: Configure S3 Event Notification

1. Open the S3 bucket
 2. Go to **Properties**
 3. Scroll to **Event Notifications**
 4. Click **Create Event Notification**
 5. Event Name: S3UploadTrigger
 6. Event Types: Select **All object create events**
 7. Destination: **Lambda Function**
 8. Select S3MetadataProcessor
 9. Save
-

Step 8: Test the Pipeline

1. Upload a file to the S3 bucket
 2. Go to CloudWatch → Log Groups → /aws/lambda/S3MetadataProcessor
 3. Verify successful execution logs
 4. Go to DynamoDB → Explore Items
 5. Confirm metadata entry exists
-

Step 9: Production Enhancements

Prevent Duplicate Entries

```
table.put_item(  
    Item=item,  
    ConditionExpression='attribute_not_exists(FileName)'  
)
```

Additional Recommendations

- Enable CloudWatch structured logging
 - Configure Dead Letter Queue (SQS)
 - Set Lambda timeout to 10–30 seconds
 - Enable DynamoDB autoscaling
-

Step 10: Security Best Practices

- Apply least privilege IAM policies
 - Enable S3 bucket encryption
 - Enable DynamoDB encryption
 - Block public access on S3
 - Enable CloudTrail logging
-

Final Validation Checklist

- ☐ DynamoDB table created
- ☐ IAM role configured
- ☐ Lambda deployed

- ☐ Environment variable configured
 - ☐ S3 event trigger configured
 - ☐ File upload tested
 - ☐ Metadata visible in DynamoDB
-

Result

You now have a fully automated, event-driven, serverless metadata ingestion pipeline using S3, Lambda, and DynamoDB.